

Course - 3

why ML strategy?

→ to know the right way to tune your model when you are not getting the desired results

Orthogonalization

Being very clear-eyed about what to tune in order to try to achieve one effect. This

process is called orthogonalization.

→ Chain of assumptions in ML

→ fit training set well on cost fn
($\approx$ human level performance)

→ fit dev set well on cost fn

→ fit test set well on cost fn

→ Performs well in real world

⇒ so in all the assumptions above you will have different knobs to adjust ~~did~~ for different assumptions. So yeah, if your model misses a particular point mentioned above you can improve it by tweaking those specific knobs

Single Number Evaluation metric

When you're tuning hyper params or trying out different ~~algs~~ options, you'll find that your progress will be much faster if you have a single real no evaluation metric, that tells you if the new thing you just tried is working better or worse than your best idea,

it's

F1 score,
it's the harmonic mean of Precision & Recall.

$$\Rightarrow \left(\frac{2}{\frac{1}{P} + \frac{1}{R}} \right) \Rightarrow \text{"Harmonic mean"}$$

Precision $\Rightarrow$ How accurate are my yeses,
 $\rightarrow$ of all the things model flagged correct how much is it actually correct

Recall $\Rightarrow$ How many real yeses did i catch
 $\rightarrow$ of all the correct examples how many did the model guessed as correct

Satisficing & Optimizing metrics

Optimizing metric $\therefore$ You want to do as well as possible on this metric

Satisficing metric $\therefore$ You setup a threshold & as long as the model is doing ~~better~~ better than threshold you are ok.

ex $\rightarrow$

Classifier	Accuracy	response time
A	90%.	150 ms
B	94%.	120 ms
C	98%.	700 ms

So, here,

Accuracy is the optimizing metric you want to do as good as possible on this one, & response time is satisficing metric, let's say threshold is (300 ms) so as long as the model is giving output in less than 300 ms we are good.

Training / dev / test distributions

→ Your dev/test/training set should come from the same distribution.

Size of the Dev & Test sets

for a small dataset, let's say 10k examples you can split the data like,

80% training set
10% dev set
10% test set

for modern deep learning large datasets, let's say 1M examples

98% training set
1% dev set
1% test set

if the size is even ~~better~~ bigger 10M, then

99% training set
0.5% dev set
0.5% test set

When to change Dev/test sets & Metrics?

Let's say you have 2 models

1	2	}	here the model 1 has 3% error but it let's through a lot of porno graphic image to user, but model 2 has 5% error but it doesn't let through pornographic images
error 3%	5%		

here, you should change the error calculation, like if a model let's pass a single image it should be assigned high cost for it

Error: $w^{(i)} = \begin{cases} 1 & \text{if } x^{(i)} \text{ is non-porn} \\ 10 & \text{if } x^{(i)} \text{ is porn} \end{cases}$

$$\frac{1}{\sum_i w^{(i)}} \sum_{i=1}^{M_{dev}} w^{(i)} L \{ y_{pred}^{(i)} \neq y^{(i)} \}$$

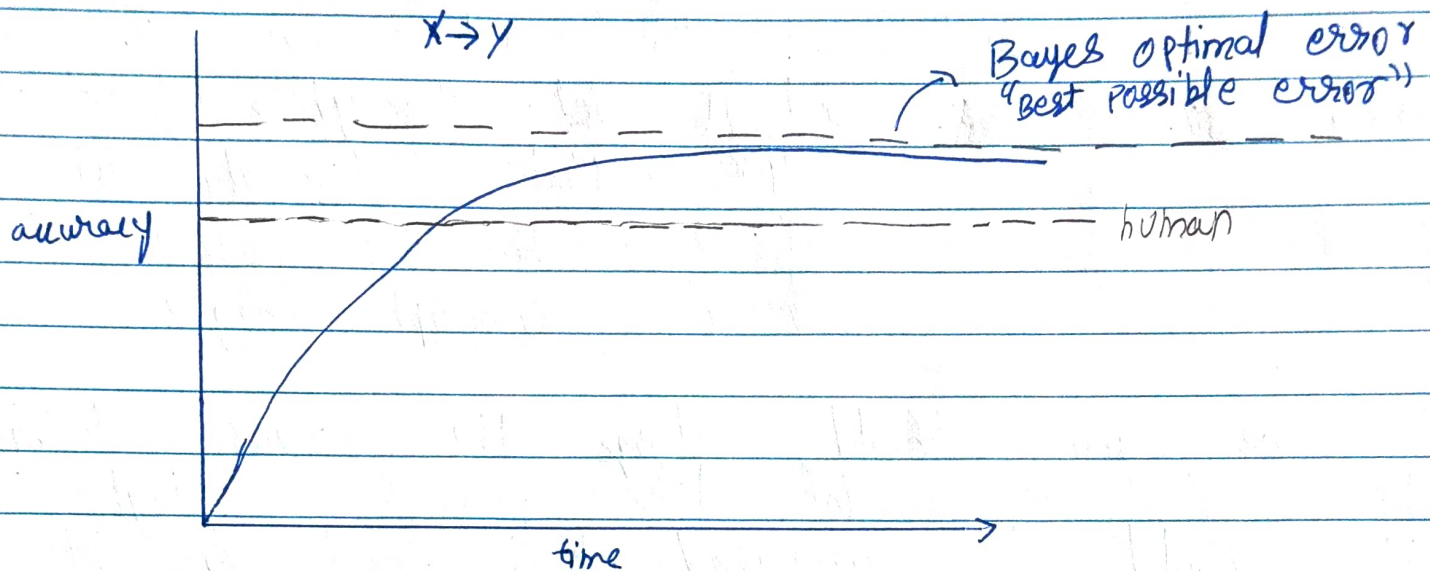
you just add a term " $w^{(i)}$ " which makes the cost bigger when the image is porn.

another example,

once you deploy the model's model 2 might perform better ~~to~~ on the data given by users because the users data are not well prepared, the images might be blurry & they might not be taken from right angle & higher quality, so it might be that model 2 performs better. so you might think of changing the test/dev set.

Why human level performance?

Comparing to human-level performance



→ You can never surpass the base optimal error

→ there is some very little improvement's you can do after reaching human level performance.

So it's good to compare to human level performance

Avoidable bias

Cat classification example

humans
training error

Dev error

①
1% $\downarrow$ 7% gap
8%
 $\uparrow$ 2% gap

10%

focus on bias

②
7.5% $\downarrow$ 0.5% gap
8%
 $\downarrow$ 2% gap

10%

focus on variance

$\Rightarrow$ in the two examples, the human level error gap between ① & ② is big so in,

① the gap is 7% meaning there is some room for improvement so you should try making the neural net as it decreases the ~~specific~~ error more compared to the gap btw training error & Dev error,

in ② the gap btw human error & training error is very low "0.5%" compared to training error "8%", so you should focus on decreasing the gap btw training error & Dev error

focus on decreasing the gap btw training error & Dev error which can be done by focusing on variance

in ② when you get this good that the difference is actually "0.5%" in humans & training error,

technically you are not fitting perfectly so there is "bias" but then again you are really close so the bias becomes avoidable & we call it "avoidable bias".

Understanding Human-level performance

having human-level error gives you the estimate for Bayes optimal error, this allows you to focus on should you try to reduce the bias or variance.

and these techniques work well until you surpass human level performance

Same as said in the avoidable bias, what should you focus on reducing bias or variance

Surpassing Human-level performance ("Ye hahahaha")

	Example 1	Example 2
Team of humans	0.5%	0.5%
One human	1%	1%
Training error	0.6%	0.3%
Dev error	0.8%	0.4%

Diagram illustrating the relationship between errors in Example 1 and Example 2:

- Example 1: Team of humans (0.5%) and One human (1%) are connected by a vertical double-headed arrow labeled 0.5%.
- Example 2: Team of humans (0.5%) and One human (1%) are connected by a vertical double-headed arrow labeled 0.5%.
- Example 1: Training error (0.6%) and Dev error (0.8%) are connected by a vertical double-headed arrow labeled 0.2%.
- Example 2: Training error (0.3%) and Dev error (0.4%) are connected by a vertical double-headed arrow labeled 0.1%.

So in let's just discard "one human" error & take "Team of humans" as an estimate of Bayes optimal error,

tell you ~~should~~ So in example one you can already work on variance, but

in example ②, the human level error is 0.5%. but the Training error is 0.3% & dev error is 0.4%. in this case you cant tell if the model is overfitting or what should you do because the model has surpassed the human level performance & you dont know what the bayes optimal error is, so yeah its kinda messed up 😞, but idk like still less gaaaaa.

Improving your model performance

Human-level



avoidable bias

→ Train Bigger model

→ Train longer/better optimization algorithms

→ NN architecture/hyperparameter search

Training error



variance

→ more data

→ Regularization

Dev error

→ NN architecture/hyperparam search

C3W2 "Structuring out Machine learning projects"

⇒ Carrying out error Analysis

→ If you're trying to get a learning algorithm to do a task that humans can do, & your learning algo is not yet at the performance of a human, Then manually examining mistakes that your algo is making, can give you insights on what to do next, ^{Process} This[^] is called error analysis

⇒ Look at dev examples to evaluate ideas

ex → let's say you have a cat classifier, that & it misclassifies some dogs as cat's

the model's accuracy → 90%.
the model's error → 10%.

Error analysis

- get ~ 100 mislabeled dev set examples
- Count up how many are dog ✓

now, suppose that 5% of your 100 mislabeled dev set examples are pictures of dogs.

So if you completely solve the dog problem your error will reduce from 10% → 9.5%, it's better to work on

Some other problem,

Suppose if it was 50/100 dog images then, it's worthwhile to solve that problem.

→ evaluate multiple ideas in parallel

Ideas for cat detection:

- fix pictures of dogs being recognized as cats
- fix great cats (lions, panthers etc...) being misrecognized
- Improve performance on blurry images.

Calculate, the no. of images in all those 3 points & optimize the one with the most no. of images, which will give you the max result

Cleaning up incorrectly labeled data

→ DL algorithms are quite robust to random errors in the training set,

it means that if you have random errors in a very small quantity in your training set it doesn't matter much,

it's not worth your time to actually go & verify each level label,

make sure your training examples don't have systematic errors.

→ if you get incorrectly labeled examples in your dev/test set, what you can do is error analysis
 ⇒ error analysis :-

model error = 10%.

Image	Dog	Grass cat	Blurry	Incorrectly labeled	Comments
98		✓			lmao
99				✓	
100			✗	✓	psuing of a cat not a real cat
% of total	8%	43%	61%	6%	

→ So, if you choose to work on Incorrectly labeled photos here well you will be improving the model's by errors
 $\frac{6}{100} \Rightarrow 6\%$ of 10
 So, 0.6%.

improvement,

So get your ass to solving something more important goofy, it ain't goin' fix shit

Correcting incorrect dev/test set examples

- Apply same process to your dev & test set's to make sure they continue to come from the same distribution.
- Consider examining examples your algorithm got right as well as ones it got wrong

→ Train & dev/test data may now come from slightly different distributions

Build your first system quickly & then iterate

- Set up dev/test set & metric
- Build initial system quickly
- Use Bias/Variance analysis & error analysis to prioritize next steps.

Training & testing on different distributions.

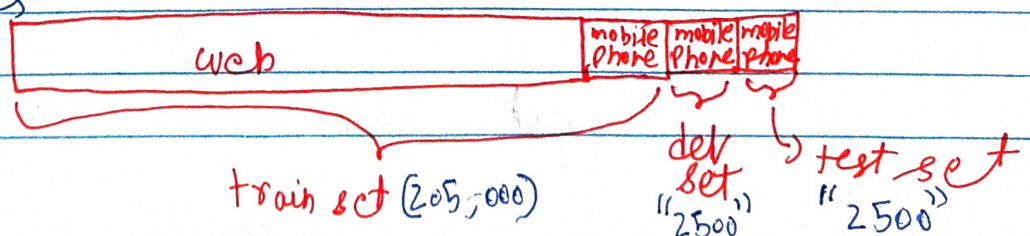
Ex →

→ if you are training cat classifier & your users are using photos from a mobile phone, but you have are training model with internet high quality images what would you do, you have very small amount of mobile phone pictures data.

Say, internet images :- 200,000
mobile phone images :- 10,000

the best thing to do is to have training set some examples of mobile phone images, & test/^{dev}~~dev~~ set only examples of mobile phone images,

it's ok if the training set & dev/test set actually come from different distribution & now you will be able to optimize for mobile phone images.



Bias & Variance with mismatched distributions

⇒ Suppose you distributed the data with only mobile phones images in dev/test set so now, how are you going to measure that,

Suppose your
training error $\rightarrow 1\%$

dev error $\rightarrow 9\%$

test error $\rightarrow 10\%$, Now how do you

know that the difference between training error & dev error is ~~not~~ a Variance problem, because your training & dev set come from different distributions,

To address this, what you can do is after making the test/dev & training set, pick a small chunk of training set & call it "train-dev" set & don't train your model on train-dev set now here are the metrics you get,

human level error $\approx 0\%$

↕ avoidable bias

Training set error $\approx 3\%$

↕ variance

Training-dev error $\approx 6\%$

↕ Data-mismatch errors

Dev error $\approx 10\%$

↕ degree of

Test error $\approx 10\%$

overfitting to dev set

Addressing data mismatch

- There isn't many things that you can do to address data mismatch but here is some example
- Carry out manual error analysis to try to understand difference between training & dev/test sets
- make training data more similar, or collect more data similar to dev/test sets

ex → You have 10 hour audio clip
You train the model with that
You test the model but error is high
You look in dev set & find out it has car noises
Now, you augment the data by editing 10 hour audio clip with car noise.
You train the model on this new data
You get better results.

basically do some data augmentation to make training set look like dev/test set, or collect more data that are like dev/test set & train model on it

Transfer learning

→ let's say you want to make an ML model on a specific task ⁴ but you don't have enough data, _{"Image task"}

what you can do is take an already trained ML model that is related to that specific task & then you get 3 cases

1. > Train only last layer :-

- only the output weights are getting updated.
- Rest of network is untouched.
- it's implemented when new data set is very small
- New task is very similar to original model task
- training whole data model with little data will cause overfitting

→

2. > Train last few layers

- Last few hidden layers + output layers updated with gradient descent
- used when data size is moderate
- Task is somewhat different

3. > Train the full neural network (full fine-tuning)

- Gradient descent updates every layer
- used when dataset is large
- New task is very different from Original
- Use a smaller learning rate for early layers.

→ larger learning rate for last layers
because early layers generally do the same
task of picking up on edges, corners & curves
so no need for big updates, but last layers
structures are different so they need to be
trained more to fit the data.

Multitask learning

→ in multitask learning, you start with a
single neural net trying to predict or do
multiple things at the same time

Ex → let's say you have a self driving simplified
car model & it's trying to predict

$y^{(1)} \rightarrow (4, 1)$
Pedestrians 0

Cars 1

Stop signs 1

Traffic lights 0

in an image

"Output labels" $Y = [y^{(1)}, y^{(2)}, y^{(3)}, y^{(4)} \dots, y^{(m)}]$

so here, Y is 2 dimensional matrix
where let's say there is an image one &
the output label to it will be a vector with
multiple 0 & 1 values,

Loss: $\hat{y}^{(i)}$
 $(1, 1)$

$$\frac{1}{M} \sum_{i=1}^M \sum_{j=1}^4 \mathcal{L}(\hat{y}_j^{(i)}, y_j^{(i)})$$

$$\mathcal{L}(\hat{y}_j^{(i)}, y_j^{(i)}) = -y_j^{(i)} \log \hat{y}_j^{(i)} - (1 - y_j^{(i)}) \log (1 - \hat{y}_j^{(i)})$$

Unlike ~~softmax~~ regression:

Here one image can have multiple labels

Multi task learning will also work if some of the labels in the image are missing for example

if

$$\begin{bmatrix} 1 \\ 0 \\ ? \\ 0 \end{bmatrix}$$

it will still work it's just that the loss will not be computed regarding the 3rd example hence there will be no gradient descent according to 3rd example hence no updates according to 3rd example

When Multitask learning makes sense

- Training on a set of tasks that could benefit from having shared lower-level features
- Usually: Amount of data you have for each task is quite similar
- Can train a big enough neural network to do well on all the tasks.

What is end to end deep learning?

End to end deep learning replaces traditional multi-step processing pipelines with one large neural net that learns to map input $\rightarrow$ output

ex $\rightarrow$ Employee face recognition.

multi-step pipelines approach

- $\rightarrow$ You take a photo
- $\rightarrow$ Cut out the face from the photo
- $\rightarrow$ feed it to neural net
- $\rightarrow$ You give that cropped image to model & then the model gives you prediction

end-to-end deep learning approach

- $\rightarrow$ You take a photo
- $\rightarrow$ give it to neural net
- $\rightarrow$ Ask for prediction's

"In the method above you generally give your model 2 images & the model has to predict those 2 images are related or not"

Pros of end to end deep learning

- 1> Let's data decide the best representation
 - > The NN learns patterns without human assumptions (like ~~ass~~ phonemes in speech)
 - Can capture better statistical structure in data.
- 2> Less manual design is required.
 - No need to hand-craft
 - features
 - Intermediate representations
 - Complex pipelines
 - simpler workflow for engineers

Cons of end to end deep learning

- 1> requires a very large data
 - needs many paired x, y examples
 - Sometimes easier to get data for subtasks than for the full task
- 2> Ignores useful human knowledge
 - Hand designed components can
 - Inject domain expertise
 - help when data is small
 - removing them may hurt performance
- 3> Not always best in real systems.
Example: Self-driving cars

- Detection $\rightarrow$ Planning $\rightarrow$ control often done in separate modules
- Pure end to end steering prediction is not so practical today.